EDUIGNITE

OPERATING SYSTEM

App Platform — Deployment Runbook

Bringing the uploadable app platform up on the server,
and getting the first app into a school’s account.

Powered by EduIgnite · Bamenda, Cameroon

1. Before you deploy

Read this section before deploying. Skipping the migration takes every feature away from every school.

1.1 The migration that matters

`registry.0002_grandfather_existing_schools` grants every existing school every non-core app.

Without it, entitlements are enforced that nobody has been granted, and every school already running loses **Attendance, Marks, Fees and Chat** the moment the release goes live. With it, the registry is a no-op for current schools and starts mattering only for new ones.

There is no safe way to deploy this release without running migrations.

1.2 New settings

All have defaults, but the first is worth setting explicitly.

Setting	Default	What it does
APP_RUNTIME_ROOT	BASE_DIR/app_runtime	Where approved builds are unpacked and packages cached. Must be writable, and
APP_PACKAGE_CACHE_ROOT	<runtime>/app_packages	Local copies of .eia files when storage is S3.
APP_RUNTIME_TIMEOUT_SECONDS	8	Wall clock per handler call.
APP_RUNTIME_MEMORY_MB	512	Address-space cap per call.
APP_RUNTIME_CPU_SECONDS	10	CPU cap per call.
APP_RUNTIME_ALLOW_INLINE	unset	Leave unset. It runs app code inside the web process, which is the thing the whole

Put `APP_RUNTIME_ROOT` somewhere nginx does not serve. If it sits under a served directory, an app's Python source becomes downloadable.

1.3 What the box needs

- **fork.** Linux is fine. Without it apps are refused rather than run unsafely, so the platform stays up but no app works.
- **Headroom for child processes.** Each in-flight call is a forked process holding up to APP_RUNTIME_MEMORY_MB. Worst case is roughly concurrent calls × that cap — check it against the instance size before enabling an app for a large school.
- **Disk** for APP_RUNTIME_ROOT: unpacked builds plus cached packages, a few MB per version, kept per version.

2. Deploying

```
# 1. Bring the code down
cd /path/to/eduignite && git pull

# 2. Dependencies, if requirements changed
source venv/bin/activate && pip install -r requirements.txt

# 3. Migrate - the step that must not be skipped
python manage.py migrate

# 4. Confirm the grandfather migration ran
python manage.py showmigrations registry

# 5. Create the runtime directory if not on the default path
mkdir -p /var/lib/eduignite/app_runtime
chown www-data:www-data /var/lib/eduignite/app_runtime

# 6. Restart all three services
sudo systemctl restart eduignite-gunicorn
sudo systemctl restart eduignite-celery-worker
sudo systemctl restart eduignite-celery-beat
```

Service names vary; use whatever this box actually calls them.

> **Restart the Celery worker and beat, not just gunicorn.** They load application code at startup. Leaving them on the previous release against the new API is the same shape as the import-prefix fault that previously put the worker into a restart loop.

2.1 Immediately after

```
sudo systemctl status eduignite-celery-worker --no-pager
sudo journalctl -u eduignite-gunicorn -n 50 --no-pager
```

Then, signed in as a founder, open **App Registry** and confirm an existing school lists its apps as installed. If a school shows nothing installed, stop and check that migration 0002 ran — before anyone notices.

3. Verifying the deploy

Do these on a **test school**, not a real one.

#	Check	Expected
1	Existing schools kept their apps	A school that existed before the release shows Attendance, Marks and Fees installed.
2	Removal works	A removed Standard app disappears from that school's menu.
3	The endpoint refuses it too	Calling the removed app's endpoint directly with a token for that school returns 403. If it returns da
4	Re-install restores everything	The school's existing records are all still there.
5	A new school arrives usable	It has the Included apps without anyone installing them.

4. First upload, end to end

4.1 Build the package

```
cd studio-main
python apps-source/teacher-attendance/build.py
# -> apps-source/teacher-attendance/dist/teacher-attendance-1.0.0.eia
```

The build is deterministic — identical source produces identical bytes — so the file can be checksummed and compared with what reached the server.

4.2 Upload

```
curl -X POST https://<host>/api/v1/registry/builds/upload/ \
  -H "Authorization: Bearer <founder token>" \
  -F "package=@apps-source/teacher-attendance/dist/teacher-attendance-1.0.0.eia"
```

Returns the build with `"state": "draft"`. **Nothing has run.** The package was validated — paths, manifest, screen bundles — and stored. A 400 says exactly what is wrong with it; that is the intended path, so fix the source and rebuild.

4.3 Submit for review

```
curl -X POST https://<host>/api/v1/registry/builds/<id>/submit/ \
  -H "Authorization: Bearer <founder token>"
```

The automated checks run immediately. Two outcomes:

- `"state": "in_review"` — passed, waiting for a person;
- `"state": "checks_failed"` — `check_report.errors` lists what failed.

The checks read the source without importing it, and catch network, filesystem, subprocess and direct database access, eval and friends, migrations with no reverse, and code performing a collection write the manifest never declared.

4.4 Approve

```
curl -X POST https://<host>/api/v1/registry/builds/<id>/approve/ \
  -H "Authorization: Bearer <reviewer token>" \
  -H "Content-Type: application/json" \
  -d '{"note": "Reviewed against the staging school."}'
```

> **The account that submitted a build cannot approve it.** That is enforced, not convention. Use a second founder account, or the approval is refused. To send it back instead, reject with a reason — a reason is required, because a rejection with none is not actionable.

4.5 Publish

```
curl -X POST https://<host>/api/v1/registry/builds/<id>/publish/ \
  -H "Authorization: Bearer <founder token>"
```

Now it is installable. The previous build, if any, becomes *deprecated* rather than deleted — schools still on it keep running, and rollback re-points at something that still exists.

4.6 Install for a school

App Registry → select the test school → **Install for this school**.

Teacher Attendance is `scope: both`, so the school licence is only half of it: each member of staff also has to add it to their own account. That is the “school purchases it, authorised users use it” case.

4.7 Open it

Sign in as a teacher at that school. **Daily Register** appears in the menu, showing every member of staff, unmarked.

Mark someone present. Then mark the same person absent. There must be **one** record for that person that day, now reading absent — a register is corrected constantly, and duplicating on correction would make the month meaningless.

5. What to watch on the first run

Symptom	Likely cause
Screen blank, spinner forever	Bundle failed to parse. Browser console inside the frame.
503 This app could not be loaded	Package not fetchable from storage. Check APP_PACKAGE_CACHE_ROOT is writable.
422 on every action	Handler raised. The real traceback is in the server log, deliberately not in the response.
422 … took too long	Handler exceeded APP_RUNTIME_TIMEOUT_SECONDS.
403 not part of your subscription	Installed for the school but not the user, or licence expired.
Apps refused platform-wide	fork unavailable. Check the log for NoIsolation.

```
sudo journalctl -u eduignite-gunicorn -f | grep -i "app \|runner\|registry"
ls -la /var/lib/eduignite/app_runtime/teacher_attendance/1.0.0/
```

6. Rolling back

One bad app, schools keep working

```
curl -X POST https://<host>/api/v1/registry/builds/<id>/pull/ \
  -H "Authorization: Bearer <founder token>" \
  -H "Content-Type: application/json" \
  -d '{"reason": "Register saves the wrong date"}'
```

Pulling stops that build running *everywhere*, including for schools already on it, and falls back to the previous published build automatically. If there is none, the app goes dark until a fix is published — the right outcome for a build doing damage.

The whole release

Revert the stage commits, redeploy, restart all three services. The registry migrations can stay: with every school grandfathered into every app, the tables are inert.

> **Do not reverse registry.0002 on a live system.** It deletes the installations that are keeping schools' features switched on.

7. Known gaps at first deploy

Stated plainly so nobody discovers them at the wrong moment.

- **Nothing here has run against a real database.** All 105 tests pass, but the scoping guarantees are verified structurally and against generated SQL, never by a real query. Section 3 is the first time that happens.
- **The fork path has never executed.** It was built on Windows, which cannot fork, so the timeout and the memory and CPU caps are untested. The timeout row in section 5 is the first real check of them.

- **No screen has rendered in a browser.** The iframe, bridge, offline cache and write queue are covered by unit tests and reading only.
- **Offline is untested end to end.** Load a screen, turn off the network, mark someone, turn it back on, confirm the change syncs.

Expect integration bugs in section 4. That is what the test school is for.